## Summary of time complexities for common Data Structures

| Data Structure | Access | Search | Insertion | Deletion |
|---|---|---|---|---|
| Array (Static) | O(1) | O(n) | N/A (fixed size) | N/A (fixed size) |
| Dynamic Array (ArrayList in Java) | O(1) (Amortized) | O(n) | O(1) (Amortized) | O(n) |
| String (Immutable in Java) | O(1) (indexing) | O(n) | O(n) | O(n) |
| Singly Linked List | O(n) | O(n) | O(1) (at head/tail) | O(1) (at head/tail) |
| Doubly Linked List | O(n) | O(n) | O(1) (at head/tail) | O(1) (at head/tail) |
| Stack (Array/Linked List based) | O(1) | O(n) | O(1) (push) | O(1) (pop) |
| Queue (FIFO - Array/Linked List based) | O(1) | O(n) | O(1) (enqueue) | O(1) (dequeue) |
| Deque (Double-ended Queue) | O(1) | O(n) | O(1) (both ends) | O(1) (both ends) |
| Priority Queue (Heap-based - Min/Max Heap) | O(n) (unsorted array) | O(n) | O(log n) | O(log n) |
| Binary Search Tree (BST) (Balanced - AVL/Red-Black) | O(log n) | O(log n) | O(log n) | O(log n) |
| Unbalanced BST (Worst Case - Skewed Tree) | O(n) | O(n) | O(n) | O(n) |
| Heap (Min-Heap / Max-Heap) | O(n) | O(n) | O(log n) | O(log n) |
| Trie (Prefix Tree) | O(m) (m = length of word) | O(m) | O(m) | O(m) |
| Hash Table (HashMap in Java) | O(1) (Average), O(n) (Worst-case w/ collisions) | O(1) (Average), O(n) (Worst-case) | O(1) | O(1) |
| Red-Black Tree (Balanced BST) | O(log n) | O(log n) | O(log n) | O(log n) |
| B+ Tree (Used in Databases, B-Trees with Leaves Linked) | O(log n) | O(log n) | O(log n) | O(log n) |

---

## Key Observations

- **Arrays** provide **O(1) random access**, but insertion/deletion is costly **O(n)**.
- **Linked Lists** offer **fast insert/delete (O(1) at head/tail)** but **slow search (O(n))**.
- **Stacks & Queues** support **O(1) insertion/deletion** at one end.
- **Heaps (Min/Max)** have **O(log n) insert/delete** due to heap property maintenance.
- **BSTs** have **O(log n) search/insert/delete** in the balanced case (**O(n) worst case** if unbalanced).
- **Red-Black Trees & AVL Trees** ensure **O(log n) for all operations**.
- **B+ Trees (used in databases)** maintain balance and are optimized for disk storage.
- **Hash Tables (HashMap)** offer **O(1) average-time operations**, but **O(n) worst-case** with collisions.

---

## Summary of Java Classes for Each Data Structure

| Data Structure | Java Class |
|---|---|
| Array | `int[]`, `Array` (Reflection), `ArrayList<E>` |
| String | `String`, `StringBuilder`, `StringBuffer` |
| List (Array & LinkedList) | `ArrayList<E>`, `LinkedList<E>` |
| Stack (LIFO) | `Stack<E>` (legacy), `ArrayDeque<E>` (recommended) |
| Queue (FIFO) | `Queue<E>`, `LinkedList<E>`, `ArrayDeque<E>` |
| Heap (Priority Queue) | `PriorityQueue<E>` (Min-Heap by default) |
| Binary Tree / BST | No built-in class (Custom Implementation) |
| Red-Black Tree | `TreeMap<K, V>` |
| B+ Tree | Not built-in (Use third-party libraries) |

Java provides built-in classes for most of these data structures in the **java.util** package. Here's a list of Java classes for each data structure:

---

## 1. Array

- **Class:** `int[]`, `Object[]`, `Array` (from `java.lang.reflect` for low-level manipulation)
- **Resizable Alternative:** `ArrayList<E>` (Dynamic array)
- **Example:**

```
int[] arr = new int[10]; // Fixed-size array
ArrayList<Integer> list = new ArrayList<>(); // Dynamic array
```

---

## 2. String

- **Class:** `String`, `StringBuilder`, `StringBuffer`
- **Example:**

```
String str = "Hello";
StringBuilder sb = new StringBuilder("Hello"); // Mutable, fast
StringBuffer sbuf = new StringBuffer("Hello"); // Thread-safe
```

---

## 3. List (Dynamic Arrays and Linked Lists)

- **Class:** `ArrayList<E>`, `LinkedList<E>`
- **Example:**

```
List<Integer> arrayList = new ArrayList<>();
List<Integer> linkedList = new LinkedList<>();
```

---

## 4. Stack (LIFO)

- **Class:** `Stack<E>` (from `java.util.Stack`)
- **Alternative (Deque is recommended):** `ArrayDeque<E>`
- **Example:**

```
Stack<Integer> stack = new Stack<>();
stack.push(10);
stack.pop();

Deque<Integer> stackDeque = new ArrayDeque<>(); // Recommended
stackDeque.push(10);
stackDeque.pop();
```

---

## 5. Queue (FIFO)

- **Class:** `Queue<E>`, `LinkedList<E>`, `ArrayDeque<E>`, `PriorityQueue<E>`
- **Example:**

```
Queue<Integer> queue = new LinkedList<>();
queue.add(10);
queue.remove();

Queue<Integer> queue2 = new ArrayDeque<>(); // Faster than LinkedList
```

---

## 6. Priority Queue (Heap - MinHeap by default)

- **Class:** `PriorityQueue<E>`
- **Example:**

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // Min-Heap
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
```

---

## 7. Trees (Binary Tree, BST, AVL, etc.)

- **No direct built-in class for general trees or BSTs**
- **Class:** `TreeMap<K, V>` (Red-Black Tree-based Sorted Map)
- **Example of a BST Implementation:**

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

---

## 8. Red-Black Tree (Balanced BST)

- **Class:** `TreeMap<K, V>` (Keys are stored in a **Red-Black Tree**)
- **Example:**

```
TreeMap<Integer, String> treeMap = new TreeMap<>();
treeMap.put(1, "One");
treeMap.put(2, "Two");
```

---

## 9. B+ Tree

- **Not available in Java's standard library**
- **Available via third-party libraries like Apache Commons DB**
- **Example:** Custom implementation or using a database index (like SQLite, MySQL)

---